

EasyFood - Passo a Passo

Arquitetura em camadas: Database, Service, Controller e Routes

Nesta aula, vamos reorganizar a arquitetura interna da EasyFood sem alterar o comportamento externo da API.

A aplicação continuará com GET /restaurants e POST /restaurants. O que muda é a forma como o código está organizado.

GET /restaurants
POST /restaurants
Prisma
PostgreSQL

Parte 1 - PREPARE

Deixe sua EasyFood no mesmo ponto da aula anterior

Antes de começar, confirme que a EasyFood já utiliza Prisma + PostgreSQL e que GET e POST estão funcionando.

```
node server.js
```

Você deverá visualizar:

EasyFood rodando na porta 3000

Teste:

```
GET http://localhost:3000/restaurants
```

Depois teste o cadastro:

```
POST http://localhost:3000/restaurants
```

```
{
  "name": "Cantina Roma",
  "category": "Italiana",
  "rating": 4.5
}
```

Se GET e POST funcionarem, continue.

Parte 2 - ENTENDA O PROBLEMA

Hoje nosso server.js concentra várias responsabilidades:

```
server.js
|
+-- inicia o Express
+-- configura middlewares
+-- define rotas
+-- valida dados
+-- acessa o Prisma
+-- conversa com o banco
+-- liga o servidor
```

Funciona, mas conforme a aplicação cresce, esse arquivo tende a ficar cada vez maior e mais difícil de manter.

Vamos separar as responsabilidades em camadas:

server.js	-> liga o servidor
app.js	-> configura a aplicação
routes	-> define os caminhos
controller	-> recebe req e devolve res
service	-> regras e operações
database	-> conexão com o Prisma
PostgreSQL	-> persistência

Parte 3 - ORGANIZE A ESTRUTURA

1. Crie as pastas

```
src/
|
+-- database/
|
+-- modules/
|
+-- restaurants/
```

macOS/Linux:

```
mkdir -p src/database src/modules/restaurants
```

No Windows, você pode criar as pastas diretamente pelo VS Code.

A estrutura final será:

```
easy-food/
|
+-- prisma/
+-- public/
+-- src/
|   +-- database/
|   |   +-- prisma.js
|   |
|   +-- modules/
|   |   +-- restaurants/
|   |       +-- restaurant.service.js
|   |       +-- restaurant.controller.js
|   |       +-- restaurant.routes.js
|   |
|   +-- app.js
|
+-- server.js
+-- package.json
+-- package-lock.json
```

Parte 4 - DATABASE

2. Isole a conexão com o Prisma

```
src/database/prisma.js

const { PrismaClient } = require("@prisma/client");

const prisma = new PrismaClient();

module.exports = prisma;
```

Responsabilidade:

Disponibilizar a conexão com o banco para o restante da aplicação.

Parte 5 - SERVICE

3. Crie o Restaurant Service

```
src/modules/restaurants/restaurant.service.js

const prisma = require("../../database/prisma");

async function listRestaurants() {
  return prisma.restaurant.findMany();
}

async function createRestaurant(data) {
  return prisma.restaurant.create({
    data: {
      name: data.name,
      category: data.category,
      rating: data.rating || 0
    }
  });
}

module.exports = {
```

```
    listRestaurants,  
    createRestaurant  
  };  
};
```

Responsabilidade:

Concentrar operações e regradas relacionadas a restaurantes.

Observe que aqui não existe req nem res. O Service não depende de HTTP.

Parte 6 - CONTROLLER

4. Crie o Restaurant Controller

```
src/modules/restaurants/restaurant.controller.js  
  
const restaurantService = require("../restaurant.service");  
  
async function list(req, res) {  
  try {  
    const restaurants = await restaurantService.listRestaurants();  
    res.json(restaurants);  
  } catch (error) {  
    res.status(500).json({  
      error: "Erro interno do servidor"  
    });  
  }  
}  
  
async function create(req, res) {  
  const { name, category, rating } = req.body;  
  
  if (!name || !category) {  
    return res.status(400).json({  
      error: "Nome e categoria são obrigatórios"  
    });  
  }  
  
  try {  
    const restaurant =  
      await restaurantService.createRestaurant({  
        name,  
        category,  
        rating  
      });  
  
    res.status(201).json(restaurant);  
  } catch (error) {  
    res.status(500).json({  
      error: "Erro interno do servidor"  
    });  
  }  
}  
  
module.exports = { list, create };
```

Responsabilidade:

Receber a requisição HTTP, validar a entrada, chamar o Service e devolver a resposta correta.

Parte 7 - ROUTES

5. Organize as rotas

```
src/modules/restaurants/restaurant.routes.js  
  
const express = require("express");  
const controller = require("../restaurant.controller");  
  
const router = express.Router();  
  
router.get("/", controller.list);  
router.post("/", controller.create);
```

```
module.exports = router;
```

Responsabilidade:

Definir os caminhos e apontar cada requisição para a função correta do Controller.

Parte 8 - APP

6. Monte a aplicação

```
src/app.js

const express = require("express");
const cors = require("cors");
const path = require("path");

const restaurantRoutes =
  require("../modules/restaurants/restaurant.routes");

const app = express();

app.use(cors());
app.use(express.json());

app.use(
  express.static(
    path.join(__dirname, "../public")
  )
);

app.use("/restaurants", restaurantRoutes);

module.exports = app;
```

Responsabilidade:

Configurar a aplicação: middlewares, JSON, arquivos estáticos e rotas.

Parte 9 - SERVER

7. Simplifique o server.js

```
const app = require("../src/app");

app.listen(3000, () => {
  console.log("EasyFood rodando na porta 3000");
});
```

Agora o server.js possui uma responsabilidade muito clara:

LIGAR O SERVIDOR

Parte 10 - TESTE A REFATORAÇÃO

8. Inicie o servidor

```
node server.js
```

9. Teste o GET

GET http://localhost:3000/restaurants

10. Teste o POST

POST http://localhost:3000/restaurants

```
{
  "name": "Cantina da Nona",
  "category": "Italiana",
  "rating": 4.7
}
```

Status esperado: 201 Created.

Se GET e POST continuam funcionando, a refatoração foi concluída sem perda de funcionalidade.

O que acabamos de fazer?

ANTES

```
server.js
|
+-- Express
+-- Rotas
+-- Validação
+-- Prisma
+-- Banco
+-- Servidor
```

AGORA

```
server.js
  |
  v
app.js
  |
  v
routes
  |
  v
controller
  |
  v
service
  |
  v
database
  |
  v
PostgreSQL
```

Mudamos a organização interna sem alterar o comportamento externo. Isso é uma refatoração.

Parte 11 - ENTENDA O FLUXO

```
REQUISIÇÃO HTTP
  |
  v
app.js
  |
  v
restaurant.routes.js
  |
  v
restaurant.controller.js
  |
  v
restaurant.service.js
  |
  v
database/prisma.js
  |
  v
PostgreSQL
```

A resposta volta pelo caminho contrário.

Parte 12 - PENSE COMO ARQUITETO

[] Por que não deixamos tudo dentro do server.js?

[] Qual é a responsabilidade do server.js?

[] Qual é a responsabilidade do app.js?

- ☐ Qual é a responsabilidade das Routes?
- ☐ Qual é a responsabilidade do Controller?
- ☐ Qual é a responsabilidade do Service?
- ☐ Qual é a responsabilidade da camada Database?
- ☐ Por que o Service não deveria depender de req e res?
- ☐ Por que as Routes não acessam diretamente o Prisma?
- ☐ O comportamento externo da API mudou?
- ☐ Que vantagem essa organização traz quando o sistema crescer?
- ☐ Que nova complexidade essa organização adiciona?

Parte 13 - MISSÃO: EVOLUA A EASYFOOD

1. ORGANIZE

Continue a modularização iniciada em aula.

```
src/  
|  
+-- modules/  
    |  
    +-- restaurants/  
    |  
    +-- auth/
```

O objetivo é pensar como o novo domínio de autenticação se encaixaria na organização criada.

2. PESQUISE

Pesquise como poderíamos implementar autenticação na EasyFood.

- ☐ JWT
- ☐ AWS Cognito
- ☐ Login com Google
- ☐ Outra solução

3. JUSTIFIQUE

Registre:

Qual solução você escolheria e por quê?

Considere o contexto atual da EasyFood e os trade-offs de cada alternativa.

DESAFIO

Conseguir implementar o login?

Show de bola.

A implementação é opcional. O mais importante é organizar, pesquisar e justificar a decisão.

Para concluir

- ☐ Estrutura em camadas criada
- ☐ database/prisma.js criado
- ☐ restaurant.service.js criado
- ☐ restaurant.controller.js criado
- ☐ restaurant.routes.js criado
- ☐ app.js criado
- ☐ server.js simplificado

- [] GET /restaurants funcionando
- [] POST /restaurants funcionando
- [] Módulo auth/ criado ou planejado
- [] Pesquisa sobre autenticação realizada
- [] Solução escolhida e justificada
- [] Opcional: login implementado

Resultado final

EasyFood

Cliente

|

v

Routes

|

v

Controller

|

v

Service

|

v

Database

|

v

PostgreSQL

Continuamos com um monólito, mas agora organizado por responsabilidades e preparado para receber novos módulos.